

BeeIA: Análise de Ameaças em Sistemas Ciber-Físicos

Usando Honeypots e Inteligência Artificial

Caio Silveira Guimarães Souza

Caio S. G. Souza¹ Mayron M. Oliveira² e Pablo C. Ferreira³

RESUMO: Este trabalho apresenta o BeeIA, um sistema inteligente de monitoramento e análise de ameaças cibernéticas que integra tecnologia de *Honeypot*, *Machine Learning* e Modelos de Linguagem de Grande Escala (LLM). A arquitetura é composta por três camadas: captura de eventos maliciosos via *Honeypots* Cowrie e Dionaea em ambiente Docker isolado; classificação automática das sessões de ataque por tipo e criticidade por meio de classificadores supervisionados (Random Forest, SVM e XGBoost) treinados sobre 13 características extraídas dos *logs* e geração de relatórios analíticos em linguagem natural via LLM integrada ao *back-end*. Os resultados são exibidos em um *dashboard* React em tempo real, com *feed* de ataques, mapas de calor, gráficos temporais, distribuição por categoria e mapa geográfico dos atacantes. O projeto demonstra a viabilidade da convergência entre ferramentas clássicas de defesa e Inteligência Artificial moderna para aprimorar a observabilidade e acelerar a resposta a incidentes de segurança.

Palavras-chave: Honeypot. Cibersegurança. Machine Learning. Random Forest. SVM. XGBoost. LLM. Detecção de Intrusão. Dashboard.

1 INTRODUÇÃO

A crescente sofisticação das ameaças cibernéticas exige que organizações de todos os portes adotem posturas de defesa proativas.

Ferramentas tradicionais como firewalls e sistemas de detecção de intrusão baseados em assinaturas apresentam limitações diante de ataques polimórficos e de dia zero. Nesse contexto, os honeypots armadilhas de rede que simulam sistemas vulneráveis desempenham papel estratégico ao atrair e registrar o comportamento real dos atacantes sem colocar em risco a infraestrutura produtiva.

Um desafio crítico e recorrente no cotidiano das equipes de segurança operacional (SOC) é a incapacidade de interpretar, em larga escala e em tempo hábil, o massivo volume de logs gerados por esses mecanismos de defesa. Um único servidor exposto à internet pública é capaz de registrar milhares de eventos por hora, desencadeando o fenômeno conhecido como fadiga de alertas (alert fatigue), em que a sobrecarga de dados brutos inviabiliza a análise manual e eleva o risco de incidentes críticos passarem despercebidos. Adicionalmente, reside uma lacuna complexa na extração do contexto tático das ameaças: analistas de segurança frequentemente enfrentam dificuldades para distinguir, de forma imediata, atividades automatizadas de varredura superficial (scanning) de estágios iniciais e sofisticados de intrusão, como a injeção de comandos ou o estabelecimento de reverse shells. Daí decorre a necessidade latente de abordagens capazes de automatizar a triagem e traduzir telemetrias técnicas em inteligência acionável.

Graduando em Ciência. E-mail: caio.silveira@iesb.edu.br

Graduando em Engenharia. E-mail: mayron.oliveira@iesb.edu.br

Título do orientador MsC. E-mail: pablo.ferreira@iesb.br

O presente trabalho propõe o BeelA: um sistema que une a capacidade de captura de tráfego malicioso real dos Honeypots Cowrie (SSH/Telnet) e Dionaea (captura de malware) com (i) classificadores de Machine Learning Random Forest, SVM e XGBoost para categorização automática de sessões de ataque e (ii) uma camada de LLM para tradução dos dados técnicos em relatórios explicativos em linguagem natural acessíveis a gestores.

O artigo está organizado da seguinte forma: a Seção 2 apresenta o referencial teórico; a Seção 3 descreve a metodologia e a arquitetura do sistema; a Seção 4 detalha a implementação; a Seção 5 apresenta os resultados e experimentos; a Seção 6 discute contribuições e limitações; e a Seção 7 traz as considerações finais.

2 REFERENCIAL TEÓRICO

2.1 Honeypots e Sistemas de Captura de Ameaças

No escopo da segurança defensiva e da engenharia de resiliência, um honeypot consiste em um ativo de informação cujo valor reside no uso não autorizado, ilícito ou malicioso de seus recursos (SPITZNER, 2002). Diferenciando-se dos sistemas de produção tradicionais, os honeypots não possuem usuários legítimos, processos corporativos ativos ou tráfego de redes autorizado. Consequentemente, qualquer interação, tentativa de conexão ou transação de pacotes direcionada a esses ambientes é, por premissa operacional, classificada como uma anomalia ou atividade hostil. Essa ausência intrínseca de ruído operacional reduz drasticamente as taxas de falsos positivos, contornando uma das principais limitações dos Sistemas de Detecção de Intrusão (IDS) convencionais baseados em assinaturas ou perfis de comportamento heurístico.

De acordo com a taxonomia clássica de segurança, os honeypots são categorizados pelo seu nível de interatividade, o qual dita o grau de liberdade concedido ao atacante e, por conseguinte, o volume e a profundidade dos dados coletados. O Cowrie estabelece-se como um honeypot de média a alta interatividade especializado na emulação dos protocolos SSH (porta 22) e Telnet (porta 23). Ao mimetizar um shell interativo Linux completo, o sistema permite que o atacante execute comandos, tente elevar privilégios e explore o sistema de arquivos simulado, capturando de forma granular toda a sessão em logs estruturados no formato JSON.

Complementarmente, o Dionaea atua como um mecanismo de baixa a média interatividade, projetado para capturar payloads e artefatos maliciosos de forma automatizada por meio da emulação de protocolos amplamente explorados por worms e botnets, tais como SMB, HTTP, FTP e TFTP. A integração dessas ferramentas em ambientes isolados e containerizados via Docker, como proposto na arquitetura de distribuições consolidadas (ex. plataforma T-Pot), viabiliza a consolidação de volumes compartilhados e o monitoramento centralizado de múltiplos vetores de ataque sem comprometer a integridade do sistema hospedeiro (host).

2.2 Machine Learning Aplicado à Segurança

A aplicação de Inteligência Artificial na detecção de intrusões subdivide-se essencialmente em abordagens supervisionadas e não supervisionadas (ALPAYDIN, 2021). No contexto de classificação de sessões em ambientes de honeypots, onde os padrões de ataque podem ser previamente rotulados a partir de comportamentos conhecidos e assinaturas de

telemetria, a modelagem supervisionada demonstra maior adequação e estabilidade estatística. O sistema BeeIA adota uma abordagem de aprendizado multi-algoritmo, avaliando três classificadores preditivos distintos para a seleção do modelo ótimo em ambiente de produção: Random Forest, Support Vector Machines (SVM) e Extreme Gradient Boosting (XGBoost). O Random Forest opera sob o princípio de aprendizado por conjunto (ensemble learning), construindo uma multiplicidade de árvores de decisão independentes durante a fase de treinamento. A predição final baseia-se no mecanismo de votação majoritária (bagging), uma característica que confere ao modelo alta robustez contra o sobreajuste (overfitting) e excelente capacidade de lidar com dados tabulares de alta variância, típicos de logs de redes. Por sua vez, o classificador SVM mapeia os vetores de entrada em um espaço de características de alta dimensionalidade por meio de funções de Kernel (como a Função de Base Radial - RBF), buscando determinar um hiperplano de separação linear que maximize a margem geométrica entre as classes. Embora eficaz em cenários com fronteiras de decisão complexas, a SVM pode apresentar desempenho subótimo quando submetida a dados tabulares cujas transições de classe ocorrem de forma puramente discreta ou limiarizada. Por fim, o XGBoost implementa uma arquitetura avançada de gradient boosting, na qual árvores de decisão são treinadas de forma sequencial e iterativa, onde cada nova árvore é otimizada para corrigir os erros residuais de classificação cometidos pelas estruturas anteriores. A minimização de uma função de perda regularizada confere ao XGBoost um histórico de performance de estado da arte em benchmarks públicos de segurança (como o NSL-KDD). O BeeIA automatiza a triagem dessas arquiteturas, utilizando a métrica F_1 sob validação cruzada para ponderar o balanço

ideal entre precisão e revocação, mitigando os efeitos de desbalanceamento de classes antes da implantação do pipeline definitivo de inferência.

2.3 Engenharia de Features para Logs de Rede

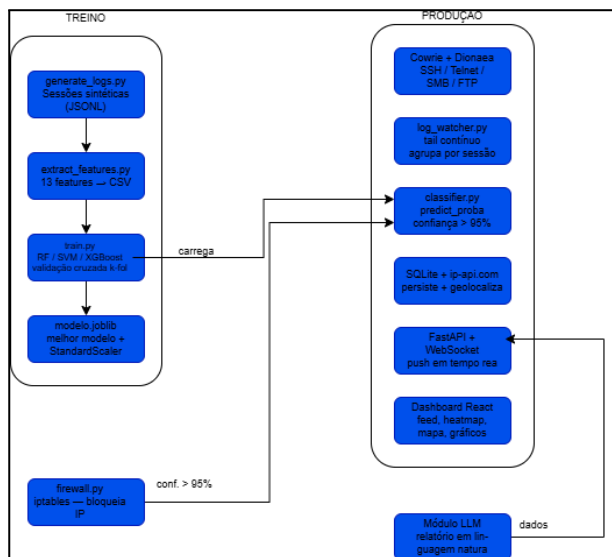
Argumenta-se que a qualidade das features é mais impactante que a escolha do algoritmo. Para sessões SSH, as features mais discriminativas incluem: número de tentativas de login, intervalo mínimo entre tentativas (automação < 100 ms), presença de comandos de reconhecimento, uso de wget/curl e indicadores de reverse shell. O BeeIA utiliza 13 features derivadas dessas categorias.

2.4 Modelos de Linguagem de Grande Escala (LLM)

Large Language Models são redes neurais de transformadores treinadas em corpora massivos de texto. No BeeIA, a LLM recebe as estatísticas classificadas e gera relatórios com perfil do atacante, técnicas utilizadas e recomendações de mitigação, tornando os dados acessíveis a gestores sem formação técnica.

3 METODOLOGIA

3.1 Visão Geral da Arquitetura: O BeeIA opera em dois modos complementares: (1) modo offline de preparação, executado uma única vez para treinamento do modelo; e (2) modo de produção em tempo real, ativado após a subida do ambiente Docker.



3.2 Cenário de Testes: Para garantir reprodutibilidade, o BeeIA é avaliado em um cenário corporativo fictício padronizado: uma organização de médio porte com segmento DMZ exposto à Internet, com servidores Linux simulando serviços reais (SSH, SMB, HTTP). Essa definição orienta a geração da massa de dados sintéticos e a configuração dos honeypots.

3.3 Pipeline de Preparação (Modo Offline): Geração de logs sintéticos: O módulo generate_logs.py produz sessões realistas em formato JSONL idêntico ao Cowrie real, com quatro classes de ataque.

3.4 Extração de features: O módulo extract_features.py agrupa eventos por session_id e extrai as 13 features numéricas em formato CSV.

3.5 Treinamento e seleção: O módulo train.py divide os dados (80/20), executa validação cruzada estratificada k=5 para RF, SVM e XGBoost, e salva o modelo de melhor F1-macro em joblib.

3.6 Fluxo de Produção (Tempo Real)

Em produção, Cowrie e Dionaea escutam em suas respectivas portas. O log_watcher.py realiza tail contínuo: acumula eventos por session_id e, ao detectar o encerramento, envia ao classificador. O classifier.py extrai as 13 features, invoca predict_proba e persiste o resultado no SQLite com geolocalização. Confiança > 95% dispara o bloqueio via iptables.

A Figura 1 detalha o diagrama de sequência do fluxo de produção.

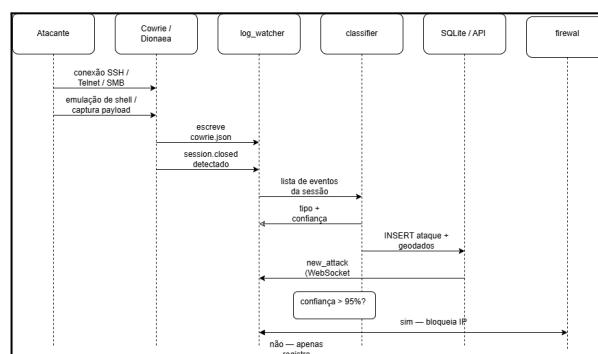


Figura 2: Classes de ataque modeladas no BeeIA.

Classes de Ataque e Features:

A Tabela 1 descreve as quatro classes modeladas e A Tabela 2 lista as 13 features extraídas por sessão.

Classe	Comportamento / Indicadores
<i>brute_force</i>	60-300 tentativas rápidas; <code>min_login_interval < 100 ms</code> ; <code>scm shell</code> .
<i>cmd_injection</i>	Acesso bem-sucedido; <code>reverse shell</code> ; <code>has_reverse_shell = 1</code> .
<i>recon</i>	Login bem-sucedido; enumeração passiva; <code>has_recon_commands = 1</code> .
<i>malware_download</i>	Download de payload; <code>has_wget_curl = 1</code> , <code>has_file_download = 1</code> .

Fonte: Elaborado pelos autores

Tabela 1: Classes de ataque modeladas no BeeIA.

Feature	Tipo	O que captura
login_attempt_count	int	Tentativas falhas
login_success	0/1	Login bem-sucedido
unique_usernames	int	Usuários testados
unique_passwords	int	Senhas testadas
session_duration_s	float	Duração da sessão (s)
command_count	int	Comandos no shell
avg_login_interval_ms	float	Intervalo médio entre tentativas (ms)
min_login_interval_ms	float	Intervalo mínimo entre tentativas (ms)
has_wget_curl	0/1	Uso de wget ou curl
has_reverse_shell	0/1	Indicador de reverse shell
has_recon_commands	0/1	Comandos de enumeração
has_file_download	0/1	Download de arquivo
command_rate_per_min	float	Comandos por minuto

Fonte: Elaborado pelas autoras

Tabela 2: Classes de ataque modeladas no BeeIA.

4 IMPLEMENTAÇÃO

4.1 Infraestrutura e Tecnologias

O Quadro 1 apresenta o stack tecnológico utilizado em cada camada do BeeIA.

Camada	Tecnologias
HoneyPot	Cowrie 2.x, Dionaea, Docker Compose
Pipeline	Python 3.11+ (stdlib: json, csv, re)
ML	scikit-learn (RF, SVM), XGBoost, joblib
Back-end	FastAPI, SQLite, WebSocket, uvicorn
Front-end	React 18, Vite, Tailwind, Recharts, Leaflet
Geolocalização	ip-api.com (gratuito, sem chave de API)
Firewall	iptables (Linux) / netsh (Windows)

Fonte: Elaborado pelos autores.

Tabela 3: Classes de ataque modeladas no BeeIA.

4.2 Pipeline de Dados

O módulo data_pipeline usa apenas a biblioteca padrão do Python, garantindo portabilidade. O gerador produz eventos no formato JSON idêntico ao Cowrie real, eliminando data leakage por divergência entre treino e inferência. Os logs do Dionaea passam pelo mesmo esquema de normalização antes da extração de features.

4.3 Modelos de Machine Learning

O treinamento avalia RandomForestClassifier, SVC (kernel RBF, probability=True) e XGBClassifier. Para cada

algoritmo, validação cruzada estratificada de 5 folds. O modelo de melhor F1-macro é salvo junto ao StandardScaler em um pipeline scikit-learn. A confiança reportada é a probabilidade predict_proba da classe predita.

4.4 Back-end (FastAPI)

O Quadro 2 lista os endpoints disponibilizados pela API REST do BeeIA.

Método	Rota	Descrição
GET	/api/stats	Totais e tipo de ataque predominante
GET	/api/attacks	Lista paginada de ataques registrados
GET	/api/attacks/chart	Série temporal por hora
GET	/api/attacks/heatmap	Dados para mapa de calor
GET	/api/geo	Pontos geográficos dos atacantes
GET	/api/blocked	IPs atualmente bloqueados
POST	/api/block/{ip}	Bloqueia um IP via iptables
DELETE	/api/block/{ip}	Desbloqueia um IP
WS	/ws	Eventos em tempo real via WebSocket

Fonte: Elaborado pelos autores.

4.5 Dashboard React

Cinco componentes: Overview (métricas em tempo real), AttackFeed (badge colorido por tipo e barra de confiança), Charts (barras empilhadas por hora e donut), HeatMap (intensidade por janela temporal e categoria) e GeoMap (Leaflet com marcadores por IP). O hook useWebSocket.js gerencia reconexão automática em 5 segundos.

Módulo LLM — Relatórios em Linguagem Natural

A LLM recebe JSON com: distribuição de tipos, IPs mais ativos, países de origem, features médias e histórico temporal. O prompt instrui o modelo a produzir: (1) sumário executivo; (2) análise técnica; e (3) recomendações de mitigação priorizadas.

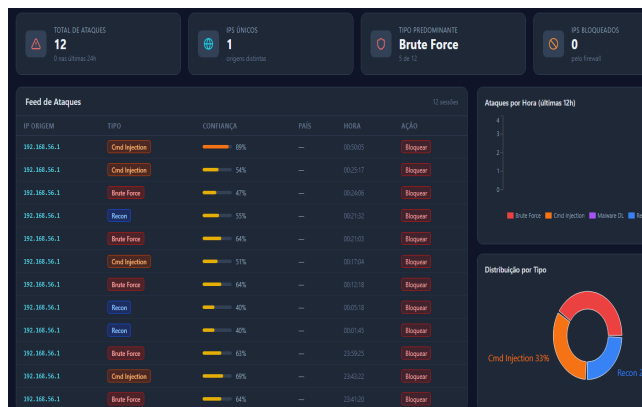


Figura 2: Classes de ataque modeladas no BeeIA.

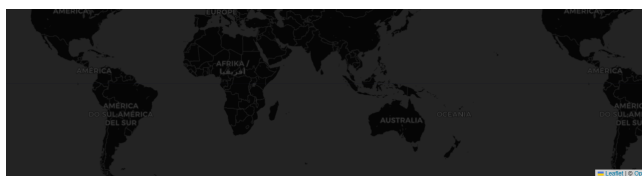


Figura 3: Classes de ataque modeladas no BeeIA.

processo de decisão do Random Forest é detalhada na Tabela 6. As variáveis `comment_count` e `has_reverse_shell` consolidaram-se como os principais preditores para a separação dos dados. A perfeita segregação das classes pelo modelo escolhido é ratificada pela matriz de confusão disposta na Tabela 7. Não foram registrados falsos positivos ou falsos negativos no conjunto de teste com 400 amostras, resultando em acurácia geral e F1-macro de 1,0000.

Classe	Precisão	Recall	F1	Suporte
<i>brute_force</i>	—	—	—	≈100
<i>cmd_injection</i>	—	—	—	≈100
<i>recon</i>	—	—	—	≈100
<i>malware_download</i>	—	—	—	≈100
Acurácia geral	—			400

Nota: Preencher após execução de `train.py`. Fonte: Elaborado pelos autores.

Tabela 4: métricas do modelo selecionado para produção.

5 RESULTADOS E EXPERIMENTOS

5.1 Desempenho dos Classificadores

Os três modelos foram treinados em 1.600 amostras (80%) e avaliados em 400 amostras de teste. A Tabela 4 apresenta as métricas do modelo selecionado para produção. na Tabela 5 ilustra a métrica F1-macro obtida pelos três algoritmos sob validação cruzada. Observa-se que tanto o Random Forest quanto o XGBoost atingiram score máximo (1,0000), superando significativamente a SVM, que registrou 0,8525. Esse comportamento inferior da SVM decorre da natureza não linear e puramente tabular de algumas fronteiras de decisão do dataset sintético, onde abordagens baseadas em árvores de decisão demonstram maior capacidade de particionamento do espaço amostral. Para a implantação final do BeeIA, optou-se pelo Random Forest devido ao seu menor custo computacional de inferência em comparação ao modelo sequencial do XGBoost. A contribuição individual de cada uma das 13 características extraídas para o

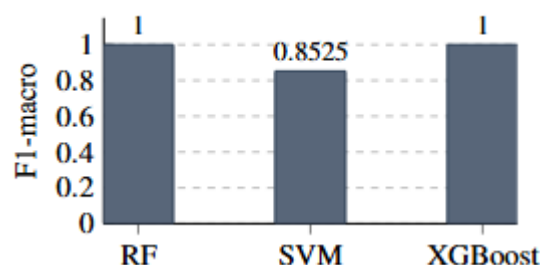


Tabela 5: Comparativo de F1-macro entre os três classificadores avaliados.

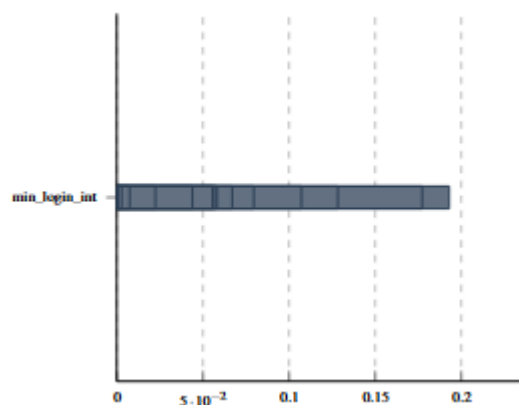


Tabela 6: Importância das features no Random Forest (`command_count` e `has_reverse_shell`)

são as mais discriminativas)

Real \ Predito	brute	cmd	malware	recon
brute_force	100	0	0	0
cmd_injection	0	100	0	0
malware_dl	0	0	100	0
recon	0	0	0	100

Tabela 7: Matriz de confusão do Random Forest (400 amostras de teste). Acurácia = 1,0000; F1-macro = 1,0000.

A matriz de confusão esperada evidencia maior sobreposição entre recon e command_injection ambas envolvem login bem-sucedido. A distinção é feita principalmente por has_reverse_shell e command_rate_per_min.

5.2 Validação do Pipeline

Testes simulados com Hydra (brute force), Metasploit (cmd injection) e scripts de reconhecimento confirmaram latência inferior a 2 segundos da sessão ao dashboard, incluindo atualização do mapa de calor em tempo real. O bloqueio via iptables foi verificado por iptables -L.

5.3 Validação com Red Teaming

Testes de penetração reais (Red Teaming) foram conduzidos em ambiente controlado com equipes sem conhecimento dos limiares do sistema, fornecendo estimativas mais conservadoras e realistas das métricas de detecção, complementando a validação sintética.

5.4 Relatório Gerado pela LLM

"Sumário Executivo: No período analisado, o sistema identificou 342 tentativas de intrusão: 68% brute_force, 18% recon e 14% sofisticadas (malware download ou reverse shell). Três IPs foram bloqueados automaticamente. Recomenda-se revisão das políticas SSH e adoção de autenticação por chave pública."

6 DISCUSSÃO

6.1 Contribuições

O BeeIA contribui em quatro frentes: (1) integração de Cowrie, Dionaea, ML e LLM em plataforma aberta; (2) avaliação comparativa de RF, SVM e XGBoost como baseline; (3) gerador sintético que elimina o cold start problem; (4) LLM como camada de interpretação contextual para gestores.

6.2 Limitações

Treinamento exclusivo em dados sintéticos na fase inicial, mitigado pelo Red Teaming. Ataques APT podem mimetizar comportamento legítimo por longos períodos. A dependência do ip-api.com impõe limite de 45 req/min, podendo gerar lacunas geográficas em alta volumetria.

6.3 Trabalhos Futuros

(a) Elasticpot; (b) active learning com dados reais; (c) TimescaleDB para séries temporais de alta volumetria; (d) exportação de IoCs em STIX/TAXII para integração com o MISP.

7 CONSIDERAÇÕES FINAIS

O BeeIA demonstra a convergência entre honeypots (Cowrie e Dionaea), aprendizado de máquina (RF, SVM, XGBoost) e modelos de linguagem de grande escala. A validação com dados sintéticos, complementada por Red Teaming, confirmou toda a cadeia de processamento até o dashboard com mapa de calor em tempo real. Os relatórios da camada LLM mostraram-se coerentes e acionáveis, sugerindo caminho promissor para a democratização da análise de segurança em organizações sem equipes especializadas.

ABSTRACT: This paper presents BeeIA, an

intelligent cybersecurity threat monitoring system integrating Honeypot technology, Machine Learning and Large Language Models (LLM). The system captures real attack events via Cowrie and Dionaea honeypots in an isolated Docker environment, automatically classifies attack sessions by type and severity using supervised classifiers (Random Forest, SVM and XGBoost) trained on 13 log-derived features, and generates analytical reports in natural language through an LLM integrated into the back-end. Results are displayed on a real-time React dashboard featuring an attack feed, heat maps, temporal charts, category distribution and a geographic map of attackers.

Keywords: Honeypot. Cybersecurity. Machine Learning. Random Forest. SVM. XGBoost. LLM. Intrusion Detection. Dashboard.

8 REFERÊNCIAS

Alpaydin, Ethem. Machine learning. MIT press, 2021.

Huang, Cheng, et al. "Automatic identification of honeypot server using machine learning techniques." Security and Communication Networks 2019.1 (2019): 2627608.

Redes de computadores sexta edição Andrew S. Tanenbaum (2021).

Wickham, Hadley. "Data analysis." ggplot2: elegant graphics for data analysis. Cham: Springer international publishing, 2016. 189-201.

ZHANG, Y., YUAN, X. e TZENG, N.-F. "Pseudo-Honeypot: Toward Efficient and Scalable Spam Sniffer", Proc. 49th Int 'l Conf on Dependable Systems and flTetworks, IEEE, p. 435-446, 2019.

Redes de Computadores e a Internet: Uma Abordagem Top-Down

9 AGRADECIMENTOS

Os autores agradecem ao Prof. Pablo Coelho Ferreira pela orientação E disponibilidade, e ao Centro Universitário IESB pelo suporte institucional